

Réalisation de l'API de Gestion Médicale (Tutoriel Détaillé)

Ce document décrit en profondeur toutes les étapes de conception, les choix architecturaux et la réalisation de l'API REST `apiv1` développée en Java 17 avec le framework Spring Boot 3.x.

Étape 1 : Initialisation du Projet Spring Boot

La commande `curl` via l'API de *Spring Initializr* a été utilisée pour générer la structure du projet.

- **Langage** : Java 17
- **Gestionnaire de paquets** : Maven
- **Packaging** : Jar

Dépendances Installées (`pom.xml`)

Voici la liste technique des dépendances Spring Boot intégrées au projet :

1. **Spring Web** (`spring-boot-starter-web`) : Intègre le serveur Tomcat et fournit les annotations (`@RestController` , `@GetMapping` , etc.) pour concevoir notre API REST.
 2. **Spring Data JPA** (`spring-boot-starter-data-jpa`) : Couche d'abstraction au-dessus de JDBC et d'Hibernate pour interagir avec la base de données via la création d'interfaces natives.
 3. **H2 Database** (`h2`) : Base de données légère qui s'exécute directement en mémoire vive (RAM). Elle est réinitialisée à chaque arrêt du serveur.
 4. **Spring Boot Validation** (`spring-boot-starter-validation`) : Permet l'usage natif de règles de contraintes (via la spécification Jakarta) telles que `@NotNull` ou `@NotBlank` sur nos objets.
 5. **Lombok** (`lombok`) : Librairie qui génère automatiquement à la compilation le code "verbeux" (*getters, setters, constructeurs, méthodes toString()*) via des annotations comme `@Data` .
-

Étape 2 : Configuration de la Base de données (H2)

La configuration d'application a été insérée dans le fichier `src/main/resources/application.properties` :

```
spring.application.name=apiv1
# URL d'accès en mémoire à H2
spring.datasource.url=jdbc:h2:mem:apiv1db
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=password

# Hibernate génère et met à jour automatiquement le schéma des tables
spring.jpa.hibernate.ddl-auto=update

# Activation de l'interface graphique accessible sur le navigateur via /h2-console
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Pourquoi `update` ? Cela permet à Hibernate de lire nos classes Java et de créer/modifier automatiquement les tables SQL, tout en conservant les données lors des recharges à chaud en développement.

Étape 3 : Implémentation de la Ressource "Patient" (Branche `api-v1`)

Nous avons adopté une **Architecture en 3 couches (MVC/API)** :

1. **Couche Entité (`@Entity`)** : `Patient.java` mappe directement les colonnes SQL. Nous utilisons `@PrePersist` pour auto-générer la date lors du premier enregistrement (`INSERT`).

```
@Entity
@Data
public class Patient {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String numero;
    private String nom;
    // ...
    @PrePersist
    protected void onCreate() {
        if(createdAt == null) createdAt = LocalDateTime.now();
    }
}
```

2. **Couche Données (`@Repository`)** : `PatientRepository.java` hérite de `JpaRepository<Patient, Long>`. Cela donne gratuitement accès à `findById()`, `save()`, `delete()`, etc.
 3. **Couche Métier (`@Service`)** : `PatientService.java` encapsule la logique métier. Exemple : génération d'un matricule automatique du type `PAT-1628189382` à la création d'un utilisateur.
 4. **Couche de Présentation (`@RestController`)** : Le `PatientController` reçoit les flux web HTTP et délègue les traitements à la couche Service.
-

Étape 4 : Ajout de la Pagination et de la Recherche

Sur un long listing de patients, il est vital de paginer. Dans `PatientRepository`, nous avons ajouté une méthode utilisant le langage **JPQL** de Spring Data pour interroger la base sans tenir compte de la casse (`LOWER()`) :

```
@Query("SELECT p FROM Patient p WHERE LOWER(p.nom) LIKE LOWER(CONCAT('%', :keyword, '%')) OR LOWER(p.prenom) LIKE LOWER(CONCAT('%', :keyword, '%'))")
Page<Patient> searchByNomOrPrenom(@Param("keyword") String keyword, Pageable pageable);
```

Dans `PatientController`, en injectant `Pageable` généré par `PageRequest.of(page, size)`, l'API retourne non plus une liste bloquante, mais un objet "Page" JSON offrant : `totalPages`, `totalElements`, et le `content`.

Étape 5 : Versionnement REST et DTOs (Branche `api-v2`)

Dès l'ouverture de la branche `v2` , toute l'ingénierie s'est renforcée :

- **Versionnement d'URL** : `/patients` devient `/api/v1/patients` .
- **Modèle DTO (Data Transfer Objects)** : Il est dangereux de renvoyer massivement les entités (exposant mots de passes, informations inutiles) ou de laisser les utilisateurs forcer un paramètre SQL.

Création de 3 sous-modèles par entité :

1. **RequestDTO** : Les informations que le client soumet. On y inclut nos validations :

```
@Data
public class PatientRequestDTO {
    @NotBlank(message = "Le nom est obligatoire")
    private String nom;
    // id ignoré, sécurité garantie
}
```

2. **ResponseDTO** : C'est ce que l'API renvoie. Toutes les données formatées.
3. **ListDTO** : Une version tronquée et très légère du `ResponseDTO` optimisée spécifiquement pour la fonction "GetAll" (sans les champs lourds tels que la description intégrale des antécédents médicaux).

Étape 6 : Mappage Manuel des DTO (`api-v2`)

Pour respecter la contrainte "*Sans MapStruct*", il a fallu coder à la main les convertisseurs sous forme de Beans Spring (`@Component`). Un `PatientMapper` a été établi pour isoler cette logique de conversion :

```
@Component
public class PatientMapper {
    public Patient toEntity(PatientRequestDTO dto) {
        // Transfert un RequestDTO en un Patient manipulable
        // (ex: setNom, setPrenom...)
    }

    public PatientResponseDTO toDto(Patient entity) {
        // Extrait les arguments du Patient venant de la base pour fabriquer le
        ResponseDTO (exposable au public)
    }
}
```

Le `PatientService` a ensuite été re-tissé pour n'accepter et ne retourner **que des DTOs**, laissant le `Controller` totalement ignorant de l'Entité H2.

Étape 7 : Ressource "Demande" et Relation `@OneToMany`

Le cœur du projet JSON-Server met en scène des **Demandes médicales liées à des Patients**.

1. **L'Entité Demande** inclut un lien vers le `Patient` géré via `@ManyToOne`. L'Enum `StatutDemande` (`en_attente`, `accepte` ...) assure qu'on ne puisse jamais glisser un statut inattendu dans la base.

```
@ManyToOne(fetch = FetchType.LAZY)
@JoinColumn(name = "patient_id", nullable = false)
private Patient patient;
```

2. **Côté Patient** (`@OneToMany`) : On ajoute la liste des demandes à l'intérieur du modèle classique :

```
@OneToMany(mappedBy = "patient", cascade = CascadeType.ALL, orphanRemoval = true)
@ToString.Exclude
@EqualsAndHashCode.Exclude
private List<Demande> demandes = new ArrayList<>();
```

! *Détail clé* : Sans `@ToString.Exclude`, Lombok aurait tenté de lire les demandes pour imprimer le patient, qui elles-mêmes auraient relu le patient indéfiniment, causant un crash fatal au lancement (`StackOverflowError`).

Étape 8 : Générateur Mock Data (Init. Base de Données)

Pour simuler un environnement réel et faciliter les tests de pagination :

- La classe `MockDataRunner` (implements `CommandLineRunner`) s'exécute quand l'application démarre.
- Un garde-fou `if(patientRepository.count() == 0)` vérifie que la base est vierge.
- L'outil injecte manuellement des faux patients (Durand, Diop, Ndiaye).
- Ensuite, il construit des requêtes (`Demande`) qu'il force à se lier à ces patients (`demande.setPatient(p1);`) et les sauvegarde.
- Grâce aux logs (ex: `System.out.println`), on peut vérifier formellement la réussite de l'injection au sein du terminal.

Exécution de l'API et Accès aux Données

Démarrage de l'Application

L'application peut être lancée grâce au *wrapper* Maven inclus dans le projet. Dans votre terminal, à la racine du projet (`apiv1/`), exécutez simplement la commande suivante :

```
./mvnw spring-boot:run
```

(Si vous construisez le projet pour la production, utilisez au préalable `./mvnw clean package` puis lancez-le avec `java -jar target/apiv1-0.0.1-SNAPSHOT.jar`)

Le serveur web embarqué Tomcat démarrera et écoutera par défaut sur le **port 8080**. Vous pourrez tester publiquement l'API via `http://localhost:8080/api/v1/patients` .

Accès à la Base de Données H2 (Console Web)

La console web de la base de données H2 a été activée dans les fichiers de configuration pour permettre la consultation en temps réel au cours du développement. Elle est accessible sur votre navigateur une fois l'application démarrée, via l'adresse :

- **Adresse internet** : <http://localhost:8080/h2-console>

Sur la page blanche de connexion, **effacez** ce qui est écrit par défaut et veuillez renseigner exactement les données de notre base :

- **Driver Class** : `org.h2.Driver`
- **JDBC URL** : `jdbc:h2:mem:apiv1db`
- **User Name** : `sa`
- **Password** : `password`

Attention: Ne laissez pas le paramètre standard `jdbc:h2:~/test` ou l'adresse du site web URL, sous peine d'erreur de *Database not found* ou d'incompatibilité de driver.

Conclusion et Sécurité Opérationnelle

En appliquant strictement cette architecture (Controller -> Service -> Repository / Entité <-> DTO), et en configurant scrupuleusement les Cascades H2, nous avons généré une API "Enterprise-Ready", modulaire (versionnée) et isolée (protégée contre la sur-exposition de données).